

地址翻译的详细文档[52]。

Knuth 在 1968 年编写了有关内存分配的经典之作[64]。从那以后，在这个领域就有了大量的文献。Wilson、Johnstone、Neely 和 Boles 编写了一篇关于显式分配器的漂亮综述和性能评价的文章[118]。本书中关于各种分配器策略的吞吐率和利用率的一般评价就引自于他们的调查。Jones 和 Lins 提供了关于垃圾收集的全面综述[56]。Kernighan 和 Ritchie [61]展示了一个简单分配器的完整代码，这个简单的分配器是基于显式空闲链表的，每个空闲块中都有一个块大小和后继指针。这段代码使用联合(union)来消除大量的复杂指针运算，这是很有趣的，但是代价是释放操作是线性时间(而不是常数时间)。Doug Lea 开发了广泛使用的开源 malloc 包，称为 dlmalloc [67]。

家庭作业

- 9.11 在下面的一系列问题中，你要展示 9.6.4 节中的示例内存系统如何将虚拟地址翻译成物理地址，以及如何访问缓存。对于给定的虚拟地址，请指出访问的 TLB 条目、物理地址，以及返回的缓存字节值。请指明是否 TLB 不命中，是否发生了缺页，是否发生了缓存不命中。如果有缓存不命中，对于“返回的缓存字节”用“-”来表示。如果有缺页，对于“PPN”用“-”来表示，而 C 部分和 D 部分就空着。

虚拟地址：0x027c

A. 虚拟地址格式

13	12	11	10	9	8	7	6	5	4	3	2	1	0

B. 地址翻译

参数	值
VPN	
TLB索引	
TLB标记	
TLB命中? (是/否)	
缺页? (是/否)	
PPN	

C. 物理地址格式

11	10	9	8	7	6	5	4	3	2	1	0

D. 物理地址引用

参数	值
字节偏移	
缓存索引	
缓存标记	
缓存命中? (是/否)	
返回的缓存字节	

- 9.12 对于下面的地址，重复习题 9.11：

虚拟地址：0x03a9

A. 虚拟地址格式

13	12	11	10	9	8	7	6	5	4	3	2	1	0

B. 地址翻译

参数	值
VPN	
TLB 索引	
TLB 标记	
TLB 命中? (是/否)	
缺页? (是/否)	
PPN	

C. 物理地址格式

11	10	9	8	7	6	5	4	3	2	1	0

D. 物理地址引用

参数	值
字节偏移	
缓存索引	
缓存标记	
缓存命中? (是/否)	
返回的缓存字节	

*9.13 对于下面的地址，重复习题 9.11:

虚拟地址: 0x0040

A. 虚拟地址格式

13	12	11	10	9	8	7	6	5	4	3	2	1	0

B. 地址翻译

参数	值
VPN	
TLB 索引	
TLB 标记	
TLB 命中? (是/否)	
缺页? (是/否)	
PPN	

C. 物理地址格式

11	10	9	8	7	6	5	4	3	2	1	0

D. 物理地址引用

参数	值
字节偏移	
缓存索引	
缓存标记	
缓存命中? (是/否)	
返回的缓存字节	

- ** 9.14 假设有一个输入文件 `hello.txt`，由字符串“`Hello, world!\n`”组成，编写一个 C 程序，使用 `mmap` 将 `hello.txt` 的内容改变为“`Jello,world!\n`”。
- * 9.15 确定下面的 `malloc` 请求序列得到的块大小和头部值。假设：1) 分配器保持双字对齐，使用隐式空闲链表，以及图 9-35 中的块格式。2) 块大小向上舍入为最接近的 8 字节的倍数。

请求	块大小 (十进制字节)	块头部 (十六进制)
<code>malloc(3)</code>		
<code>malloc(11)</code>		
<code>malloc(20)</code>		
<code>malloc(21)</code>		

- * 9.16 确定下面对齐要求和块格式的每个组合的最小块大小。假设：显式空闲链表、每个空闲块中有四字节的 `pred` 和 `succ` 指针、不允许有效载荷的大小为零，并且头部和脚部存放在一个四字节的字中。

对齐要求	已分配块	空闲块	最小块大小 (字节)
单字	头部和脚部	头部和脚部	
单字	头部，但是没有脚部	头部和脚部	
双字	头部和脚部	头部和脚部	
双字	头部，但是没有脚部	头部和脚部	

- ** 9.17 开发 9.9.12 节中的分配器的一个版本，执行下一次适配搜索，而不是首次适配搜索。
- ** 9.18 9.9.12 节中的分配器要求每个块既有头部也有脚部，以实现常数时间的合并。修改分配器，使得空闲块需要头部和脚部，而已分配块只需要头部。
- * 9.19 下面给出了三组关于内存管理和垃圾收集的陈述。在每一组中，只有一句陈述是正确的。你的任务就是判断哪一句是正确的。
- 1) a) 在一个伙伴系统中，最高可达 50% 的空间可以因为内部碎片而被浪费了。
b) 首次适配内存分配算法比最佳适配算法要慢一些 (平均而言)。
c) 只有当空闲链表按照内存地址递增排序时，使用边界标记来回收才会快速。
d) 伙伴系统只有有内部碎片，而不会有外部碎片。
- 2) a) 在按照块大小递减顺序排序的空闲链表上，使用首次适配算法会导致分配性能很低，但是可以避免外部碎片。
b) 对于最佳适配方法，空闲块链表应该按照内存地址的递增顺序排序。
c) 最佳适配方法选择与请求段匹配的最大的空闲块。
d) 在按照块大小递增的顺序排序的空闲链表上，使用首次适配算法与使用最佳适配算法等价。
- 3) Mark & Sweep 垃圾收集器在下列哪种情况下叫做保守的：
a) 它们只有在内存请求不能被满足时才合并被释放的内存。
b) 它们把一切看起来像指针的东西都当做指针。
c) 它们只在内存用尽时，才执行垃圾收集。
d) 它们不释放形成循环链表的内存块。

**** 9.20** 编写你自己的 malloc 和 free 版本，将它的运行时间和空间利用率与标准 C 库提供的 malloc 版本进行比较。

练习答案

9.1 这道题让你对不同地址空间的大小有了些了解。曾几何时，一个 32 位地址空间看上去似乎是无法想象的大。但是，现在有些数据库和科学应用需要更大的地址空间，而且你会发现这种趋势会继续。在有生之年，你可能会抱怨个人电脑上那狭促的 64 位地址空间！

虚拟地址位数 (n)	虚拟地址数 (N)	最大可能的虚拟地址
8	$2^8 = 256$	$2^8 - 1 = 255$
16	$2^{16} = 64K$	$2^{16} - 1 = 64K - 1$
32	$2^{32} = 4G$	$2^{32} - 1 = 4G - 1$
48	$2^{48} = 256T$	$2^{48} - 1 = 256T - 1$
64	$2^{64} = 16\ 384P$	$2^{64} - 1 = 16\ 384P - 1$

9.2 因为每个虚拟页面是 $P=2^p$ 字节，所以在系统中总共有 $2^n/2^p=2^{n-p}$ 个可能的页面，其中每个都需要一个页表条目(PTE)。

n	$P=2^p$	PTE的数量
16	4K	16
16	8K	8
32	4K	1M
32	8K	512K

9.3 为了完全掌握地址翻译，你需要很好地理解这类问题。下面是如何解决第一个子问题：我们有 $n=32$ 个虚拟地址位和 $m=24$ 个物理地址位。页面大小是 $P=1KB$ ，这意味着对于 VPO 和 PPO，我们都需要 $\log_2(1K)=10$ 位。(回想一下，VPO 和 PPO 是相同的。)剩下的地址位分别是 VPN 和 PPN。

P	VPN位数	VPO位数	PPN位数	PPO位数
1KB	22	10	14	10
2KB	21	11	13	11
4KB	20	12	12	12
8KB	19	13	11	13

9.4 做一些这样的手工模拟，能很好地巩固你对地址翻译的理解。你会发现写出地址中的所有位，然后在不同的位字段上画出方框，例如 VPN、TLBI 等，这会很有帮助。在这个特殊的练习中，没有任何类型的不命中：TLB 有一份 PTE 的副本，而缓存有一份所请求数据字的副本。对于命中和不命中的一些不同的组合，请参见习题 9.11、9.12 和 9.13。

A. 00 0011 1101 0111

B.

参数	值
VPN	0xf
TLB索引	0x3
TLB标记	0x3
TLB命中? (是/否)	是
缺页? (是/否)	否
PPN	0xd

C. 0011 0101 0111